Software Requirements Specification (SRS)

Infinity Blog - Content Management System

Document version: 1.0

Status: Baseline for implementation

Technology: PHP and MySQL

Academic context: BCA mini-project

Document control

Item	Value
System name	Infinity Blog
Document	Software Requirements Specification
Version	1.0
Status	Initial requirements baseline
Primary users	Public visitors, authors, administrators
Core technology	PHP, MySQL

Contents

- 1. Introduction
- 2. Overall description
- 3. Scope
- 4. Functional requirements
- 5. Use cases
- 6. Data requirements
- 7. External interface requirements
- 8. Non-functional requirements
- 9. Security constraints and business rules
- 10. Acceptance criteria
- 11. Requirements traceability
- 12. Future enhancements

Contents are section-based; page numbers may change as the specification is revised.

1. Introduction

1.1 Purpose

This document defines the functional and non-functional requirements for Infinity Blog, a web-based Content Management System. It establishes what the system must do, what information it stores, how users interact with it, and how the implementation will be verified.

1.2 Official problem statement

A blog site requires a CMS that allows authors to write, edit, and publish articles. The system must use PHP and MySQL, allow authors to log in, create new posts, edit existing posts, and delete posts, and display posts to the public in a structured format.

The requirements in this document preserve that core scope. Administration, category assignment, and AI article summarization are defined as additional agreed project requirements.

1.3 Objectives

- Provide secure sign-in for authors and administrators.
- Allow authors to create, edit, publish/unpublish, and delete their own posts.
- Allow administrators to manage users, all posts, and categories.
- Allow authors to assign available administrator-created categories to posts.
- Display published posts to the public in a structured format.
- Support keyword search and category filtering of published posts.
- Generate and store article summaries through a controlled AI service when requested.
- Protect the system against common web attacks through enforceable application controls and testing.

1.4 Intended audience

Student developer(s), faculty reviewer, frontend developer, backend developer, tester, and maintainer.

1.5 Definitions

Term	Definition
CMS	Content Management System
Author	Authenticated user allowed to manage their own posts
Admin	Authenticated administrator with system-wide management permissions
Draft	Post not visible to the public
Published	Post approved for public display under the application's publication rules
Active category	Category allowed for new assignments/publication
Summary	Plain-text AI-generated representation of an article
Stale summary	Summary generated from an earlier version of article title/body
CSRF	Cross-Site Request Forgery
XSS	Cross-Site Scripting
IDOR	Insecure Direct Object Reference / broken object-level authorization

2. Overall description

2.1 Product perspective

Infinity Blog is a PHP application backed by MySQL. The frontend uses a documented contract to communicate with server-side routes/API endpoints. Server-side code is responsible for authentication, authorization, validation, database operations, summary generation, and public visibility rules. The AI provider is an optional external dependency and must not be required for ordinary CMS operations.

2.2 User classes

Public visitor
Can view published posts, search, filter by active category, and open article details. Does not need an account.

Author
Can sign in, create drafts, edit and delete permitted posts, select an active category, publish/unpublish own posts, and request a summary for a post they may manage.

Administrator
Can sign in to the Admin Panel, manage all posts, manage author accounts, create/edit/deactivate categories, and review permitted audit events. Administrative routes require an admin role checked server-side.

2.3 Initial administrators
The system must support secure provisioning of these administrator identifiers:

- `Shubham Banerjee`
- `Sounak Sikdar`

Passwords must be configured securely during provisioning, hashed before storage, not hardcoded in application code or source control, and preferably distinct for each administrator. No plaintext credential is part of this specification.

2.4 Operating environment
- A supported PHP 8.x runtime.
- MySQL 8.x or a compatible supported database release.
- Apache or an equivalent supported web server.
- Modern browser for the frontend.
- HTTPS in production.
- A configured AI provider is required only for summary generation.

2.5 Assumptions and dependencies
- The hosting environment can run PHP and connect securely to MySQL.
- A provider account/key may be configured for the AI feature; without it, summary generation reports a safe unavailable state while core CMS operations continue.
- Initial administrator accounts are created via a trusted setup/CLI flow, not public self-registration.
- Public visitors never receive access to drafts or administrative data.
- The actual repository must be inspected before choosing libraries or changing working code.

3. Scope

3.1 In scope
- Login/logout and session management.
- `admin` and `author` roles.
- Author dashboard and admin dashboard.
- Author and admin dashboards show total, draft, and published post counts calculated from database records and scoped by role.
- Post creation, editing, publishing/unpublishing, deletion, and listing.
- Admin user management with safe account activation/deactivation.
- Admin-only category management.
- Author assignment of active categories.
- Public published-post listing, detail, search, and category filtering.
- AI-generated article summary stored in the database and reused on public views.
- Audit logging for important administrative/content events.
- Input validation, output encoding, CSRF protection, prepared SQL, authorization checks, rate limiting, safe error handling, and testing.

3.2 Out of scope for Version 1.0

- Comments, likes, followers, direct messaging, or forum functionality.
- E-commerce, subscription billing, advertisements, or payments.
- Campus ERP or student-management functionality.
- Native mobile app.
- Multi-tenant organizations.
- AI browsing, tools, code execution, database querying, or autonomous actions.
- View tracking and view-count metrics.
- Post tags.
- Featured-image uploads and other user file uploads.
- Public author profile pages.
- Remember-me login sessions.
- Password recovery/reset workflows.
- Trash/restore workflows. Post deletion is permanent after explicit UI confirmation.

4. Functional requirements

Each requirement is testable and identified with an ID.

4.1 Authentication and sessions

ID	Requirement	Priority
FR-AUTH-01	The system shall allow registered, active users to submit an identifier and password for authentication.	Must
FR-AUTH-02	The system shall verify a submitted password against a stored password hash; it shall not store plaintext passwords.	Must
FR-AUTH-03	The system shall display a generic authentication failure message without revealing whether an identifier exists.	Must
FR-AUTH-04	The system shall regenerate the session ID after successful authentication.	Must
FR-AUTH-05	The system shall invalidate the server session on logout.	Must
FR-AUTH-06	The system shall block unauthenticated access to author/admin management routes.	Must
FR-AUTH-07	The system shall apply login rate limits or equivalent brute-force controls.	Must
FR-AUTH-08	The system shall prevent inactive accounts from using protected capabilities.	Must

4.2 Role-based authorization

ID	Requirement	Priority
FR-ACL-01	The system shall support at least admin and author roles.	Must
FR-ACL-02	The system shall enforce roles on the server for every protected endpoint.	Must
FR-ACL-03	The system shall derive the acting user identity from the authenticated session, not a client-supplied author ID.	Must
FR-ACL-04	An author shall only manage posts they own unless an explicitly authorized admin policy applies.	Must
FR-ACL-05	An author shall not access admin-only user or category mutation endpoints.	Must

ID	Requirement	Priority
FR-ACL-06	Public endpoints shall not expose drafts, private management fields, password hashes, or secrets.	Must

4.3 Post management

ID	Requirement	Priority
FR-POST-01	An authenticated author shall be able to create a post with a title and content.	Must
FR-POST-02	The system shall associate a post with the authenticated author.	Must
FR-POST-03	The system shall validate required post fields and enforce length limits server-side.	Must
FR-POST-04	An authorized author/admin shall be able to retrieve a post for management.	Must
FR-POST-05	An author shall be able to edit a permitted post and persist changes without creating a duplicate record.	Must
FR-POST-06	An authorized author/admin shall be able to delete a permitted post after a confirmation interaction in the frontend.	Must
FR-POST-07	The system shall support at least draft and published post states.	Must
FR-POST-08	An authorized author/admin shall be able to publish a valid post.	Must
FR-POST-09	An authorized author/admin shall be able to unpublish a post.	Must
FR-POST-10	The system shall record creation, update, and publication timestamps.	Must
FR-POST-11	The system shall list an author's manageable posts in the author area.	Must
FR-POST-12	The system shall provide admins with a system-wide post list.	Must
FR-POST-13	The system should list recent posts first unless another explicit sort is chosen.	Should

4.4 Category management and assignment

ID	Requirement	Priority
FR-CAT-01	The system shall store categories separately from posts and connect them through a category ID.	Must
FR-CAT-02	Only administrators shall be able to create categories.	Must
FR-CAT-03	Only administrators shall be able to edit, activate, or deactivate categories.	Must
FR-CAT-04	Category names shall be unique. The server shall generate each URL-safe category slug from its name and enforce slug uniqueness in application validation and database constraints.	Must
FR-CAT-05	Authors shall be able to select an existing active category for a post.	Must
FR-CAT-06	The backend shall reject nonexistent or inactive category IDs when a post is assigned/published under the applicable rules.	Must

ID	Requirement	Priority
FR-CAT-07	A post shall require a valid active category before publication. Drafts may remain uncategorized if allowed by the implementation.	Must
FR-CAT-08	Deactivating a category shall preserve existing post relationships and shall prevent new assignment of that category.	Must
FR-CAT-09	Public visitors shall be able to filter published posts by category.	Must
FR-CAT-10	The system shall not provide authors with category mutation privileges.	Must

4.5 Public article discovery and display

ID	Requirement	Priority
FR-PUB-01	The public homepage shall list published posts in a structured format.	Must
FR-PUB-02	The system shall provide a public detail page/response for each published post.	Must
FR-PUB-03	Public post representations should include title, author display name, category where assigned, publication date, and excerpt/summary when current.	Should
FR-PUB-04	The system shall never return draft content through public list, detail, search, or category routes.	Must
FR-PUB-05	The system shall support keyword search across permitted public fields.	Should
FR-PUB-06	The system should support pagination with a bounded maximum page size.	Should
FR-PUB-07	The public article page shall render stored summary text safely when the summary is current.	Must
FR-PUB-08	If the summary is stale or unavailable, the public page shall not present it as current; it may display an excerpt or omit the summary.	Must

4.6 Admin panel and user management

ID	Requirement	Priority
FR-ADM-01	The system shall provide a protected Admin Panel.	Must
FR-ADM-02	The admin dashboard shall display system-wide total, draft, and published post counts calculated from database records.	Must
FR-ADM-03	An administrator shall be able to view and manage all posts under the defined policy.	Must
FR-ADM-04	An administrator shall be able to create/provision author accounts through an authorized workflow.	Must
FR-ADM-05	An administrator shall be able to activate/deactivate author accounts.	Must
FR-ADM-06	Role changes shall be restricted to explicitly authorized operations and audited.	Must
FR-ADM-07	The system shall never expose password hashes or credentials in user-management responses.	Must

ID	Requirement	Priority
FR-ADM-08	The system shall provide an admin-only view of permitted audit events.	Should
FR-ADM-09	The system shall support secure provisioning of the two specified initial admin identities without storing a plaintext default password in source control.	Must
FR-ADM-10	The admin dashboard should display user and category totals where those values are available from existing admin data.	Should
FR-DASH-01	The author dashboard shall display total, draft, and published counts for posts owned by the authenticated author, calculated from database records.	Must
FR-DASH-02	Dashboard counts shall not include view counts or expose another author's private posts or counts to an author.	Must

4.7 Article summarization

ID	Requirement	Priority
FR-AI-01	An authorized author/admin shall be able to request a summary for a post they are permitted to manage.	Must
FR-AI-02	The backend shall load canonical article title/body from the database after authorization.	Must
FR-AI-03	Summary generation shall occur only on an explicit authorized request, not on every public article view.	Must
FR-AI-04	The system shall store the summary and its status, source marker/hash, and generation timestamp where applicable.	Must
FR-AI-05	The system shall treat article content as untrusted input that may contain prompt-injection instructions.	Must
FR-AI-06	The provider adapter/model shall receive no database credentials, tools, or application privileges; only the authorized backend operation may load and persist the post through the normal data-access layer.	Must
FR-AI-07	The system shall validate provider output and store/render summary as plain text in Version 1.0.	Must
FR-AI-08	The system shall enforce request timeout, input/output size limits, and rate limiting for summary generation.	Must
FR-AI-09	A title/body edit shall mark an existing summary stale.	Must
FR-AI-10	AI provider failure shall not prevent normal post creation, editing, publishing, or reading.	Must
FR-AI-11	The provider key and model configuration shall remain server-side.	Must
FR-AI-12	The system shall display a safe error when summarization is unavailable or rate-limited.	Must

4.8 Audit and error handling

ID	Requirement	Priority
FR-LOG-01	The system should audit post publish/unpublish/delete, category changes, user activation/role changes, and summary outcomes.	Should
FR-LOG-02	Audit logs shall not record passwords, session cookies, API keys, full provider secrets, or full article bodies.	Must
FR-ERR-01	The system shall return meaningful validation errors for invalid requests.	Must
FR-ERR-02	The system shall return generic production errors and keep stack traces/SQL diagnostics private.	Must
FR-ERR-03	A failed write shall not be reported to the user as successful.	Must

5. Use cases

UC-01 - Author login

Actor: Author

Precondition: Account exists and is active.

Main flow: User submits identifier/password; server rate limit and credential check run; the password hash is verified; session ID is regenerated; role and safe identity are returned; frontend opens the correct dashboard.

Alternate flow: Invalid or inactive account is rejected using safe messaging.

Postcondition: Authenticated session exists only after a successful check.

UC-02 - Create a draft

Actor: Author

Precondition: Author is authenticated.

Main flow: Author enters title/body, optionally selects category, chooses draft, submits; server validates fields, derives author from session, stores the post, and returns the created post.

Postcondition: Draft is saved and not public.

UC-03 - Publish a post

Actor: Author or admin with permission

Precondition: Post exists and requester may manage it.

Main flow: Requester chooses Publish; server verifies title/body and a valid active category; status and publication timestamp are stored; an audit event is written.

Alternate flow: Invalid/inactive category or incomplete content causes a validation failure.

Postcondition: Post appears on public list/detail routes.

UC-04 - Edit an existing post

Actor: Author or admin with permission

Precondition: Requester is authorized for the post.

Main flow: Server loads permitted post; requester submits edits; server validates and updates the same record; if title/body changed, existing summary is marked stale.

Postcondition: Updated data is stored and authorization remains intact.

UC-05 - Delete a post

Actor: Author owner or admin

Precondition: Requester is authorized.

Main flow: Frontend asks for confirmation; request contains valid CSRF token; server rechecks authorization and permanently deletes the post; an audit event is stored.

Postcondition: Post is no longer accessible through public routes.

UC-06 - Create a category

Actor: Admin

Precondition: Admin is authenticated and active.

Main flow: Admin submits name/description; server validates admin role and name uniqueness, generates a normalized URL-safe slug with a numeric suffix on collision, and stores the category active.

Alternate flow: Duplicate normalized category name is rejected. A concurrent slug collision is retried under the database uniqueness constraint.

Postcondition: Active category is available to author forms.

UC-07 - Assign category to a post

Actor: Author

Precondition: Author is authorized for the post and category is active.

Main flow: Author selects category; server validates ID/state and stores category relationship.

Postcondition: Post is associated with the selected category. Authors cannot create or mutate the category itself.

UC-08 - Generate article summary

Actor: Author owner or admin

Precondition: Requester may manage post; summarization is configured and request is under limit.

Main flow: Server loads the current article, sends bounded untrusted text to configured provider with fixed summarization instructions, validates plain-text response, stores summary/hash/timestamp, and returns status.

Alternate flow: Missing provider config, rate limit, timeout, invalid response, or provider error returns a safe error and preserves the article.

Postcondition: Current summary is stored or a failure/stale state is recorded.

UC-09 - Browse public articles

Actor: Public visitor

Precondition: None.

Main flow: Visitor opens homepage; server returns published posts only; visitor optionally searches/filters; visitor opens an article detail.

Postcondition: Only published content is exposed.

6. Data requirements

6.1 Users

Store user ID, unique login identifier, display name, optional email, password hash, role, active state, password-change flag if required, creation/update timestamps, and optional last-login timestamp. Do not expose password hashes.

6.2 Categories

Store category ID, unique name, unique server-generated slug, optional description, active state, creator, and timestamps. Admin-only mutation is enforced in the backend. Category create/edit requests do not accept a client-authoritative slug.

6.3 Posts
Store post ID, author ID, category ID (nullable only while draft if allowed), title, body, status, summary, summary status, summary source hash, summary generation timestamp, publication timestamp, creation timestamp, and update timestamp.

6.4 Audit logs
Store actor, action, entity type/ID, safe metadata, and timestamp. Avoid storing secrets or full article/provider payloads.

6.5 Integrity rules
- Primary keys are required.
- Posts reference valid authors.
- Posts reference valid categories where assigned.
- Category deactivation preserves existing references.
- Public queries filter by published status.
- Required status values are allowlisted and enforced server-side.

7. External interface requirements

7.1 User interface
The user interface shall provide:

- Login page.
- Author dashboard and post forms/list.
- Admin dashboard, user management, all-post management, category management, and permitted activity view.
- Public blog list, search/filter, and article detail.
- Summary generation action and summary status/preview where allowed.
- Loading, empty, validation error, forbidden, not-found, rate-limited, and server-error states.

Visual appearance is specified separately and is not fixed by this SRS.

7.2 API
The frontend/backend interface is defined in `API_SPEC.md`. It shall use consistent success/error response structures, validate inputs, return appropriate status codes, and never expose internals in errors.

7.3 Database interface
PHP shall communicate with MySQL through a centralized data-access approach using prepared statements. Foreign keys and indexes shall support data integrity and common queries.

7.4 AI interface
The AI provider shall be called only by the backend via a replaceable provider adapter. Secrets remain server-side. Calls must be bounded and must not grant tools or application privileges to the model.

8. Non-functional requirements

8.1 Security
- NFR-SEC-01: Passwords shall be hashed using an approved password-hashing function.
- NFR-SEC-02: All user-influenced SQL values shall use prepared/parameterized statements.
- NFR-SEC-03: Protected routes shall enforce authentication, roles, and resource ownership server-side.
- NFR-SEC-04: State-changing browser operations shall validate CSRF tokens.
- NFR-SEC-05: Untrusted output shall be encoded for its rendering context; summaries are plain text.
- NFR-SEC-06: Production cookies shall use secure flags and HTTPS shall be enforced.
- NFR-SEC-07: Secrets shall not be stored in version control or returned to clients.
- NFR-SEC-08: Login and summary generation shall be rate-limited.
- NFR-SEC-09: AI provider failures and untrusted article instructions shall not grant the model application authority.
- NFR-SEC-10: Production errors shall not disclose stack traces, query details, filesystem paths, or secrets.

8.2 Performance

- NFR-PERF-01: Normal CMS operations should complete without unnecessary external service calls.
- NFR-PERF-02: Public article reads shall not synchronously generate a summary.
- NFR-PERF-03: Public and admin lists shall use bounded pagination.
- NFR-PERF-04: AI calls shall have a configured timeout and bounded payload size.

8.3 Reliability

- NFR-REL-01: Failed AI generation shall not corrupt post data.
- NFR-REL-02: Database write failures shall be reported safely and shall not be misrepresented as success.
- NFR-REL-03: Schema migrations and backup/restore procedures shall be documented.

8.4 Maintainability

- NFR-MAINT-01: Authentication, authorization, post logic, category logic, and summary integration shall have clear module boundaries.
- NFR-MAINT-02: API and schema changes shall be reflected in documentation and tests.
- NFR-MAINT-03: Dependencies shall be limited and kept reasonably current.

8.5 Usability and compatibility

- NFR-UX-01: Required author actions shall have clear labels and visible success/error feedback.
- NFR-UX-02: Destructive actions shall require confirmation in the UI, with authorization rechecked server-side.
- NFR-UX-03: Public article content shall be readable and structured.
- NFR-COMP-01: The frontend shall work in supported modern browsers at common desktop and mobile viewport sizes.

9. Security constraints and business rules

1. Only admin can create/edit/activate/deactivate categories.
2. Authors can select only active categories.
3. An active category is required before publication.
4. Only published posts are public.
5. Authors can only modify their own posts unless acting under an explicit admin permission.
6. Admin setup passwords are provisioned securely and never stored as plaintext.
7. Summaries are stored and reused; public reads never trigger AI calls.
8. Summary output is untrusted plain text.
9. The provider adapter receives bounded article text only and has no database credentials, tools, or application privileges; the authorized backend operation performs database access.
10. A stale summary cannot be presented as current.
11. Category deactivation preserves existing post links.
12. User IDs, roles, and ownership cannot be trusted from request bodies.

10. Acceptance criteria

- AC-01: Valid author login opens author dashboard; invalid credentials are rejected safely.
- AC-02: Admin login opens Admin Panel and every admin route rejects authors.
- AC-03: An author can create and edit own posts.
- AC-04: An author cannot edit or delete another author's post by changing IDs.
- AC-05: An author can publish a post only when publication validation succeeds.
- AC-06: Public routes return published posts only.
- AC-07: Admin can create a category; an active category is available to authors.
- AC-08: An author cannot create, rename, activate, deactivate, or delete a category.
- AC-09: Deactivating a category preserves existing references and prevents new assignments.
- AC-10: Public category filtering and search return only permitted published posts.
- AC-11: An authorized summary request stores validated plain-text output and status metadata.
- AC-12: Public article views do not call the AI provider.
- AC-13: Changing article title/body marks an existing summary stale.
- AC-14: Provider timeout/failure does not break core post operations.

- AC-15: CSRF, injection, XSS, session, rate-limit, and authorization checks in `TEST_PLAN.md` pass before release.
- AC-16: No credentials or provider keys are committed or exposed through API responses/logs.
- AC-17: Author and admin post counts match database records within their authorized scope; authors cannot infer other authors' private post counts and neither dashboard includes view counts.
- AC-18: Server-generated category slugs remain unique when different names normalize to the same base slug; duplicate normalized category names are rejected.
- AC-19: AI-provider failure does not prevent post creation, editing, or publication; public reads reuse current summaries stored in MySQL.

11. Requirements traceability

Problem statement / product need	Requirements
Author login	FR-AUTH-01 through FR-AUTH-08
Create articles	FR-POST-01 through FR-POST-03
Edit existing posts	FR-POST-04, FR-POST-05
Publish and delete	FR-POST-06 through FR-POST-10
Public structured display	FR-PUB-01 through FR-PUB-08
Admin panel	FR-ADM-01 through FR-ADM-10
Admin creates categories, authors assign them	FR-CAT-01 through FR-CAT-10
AI summaries	FR-AI-01 through FR-AI-12
Safe implementation	FR-ACL, FR-ERR, NFR-SEC, SECURITY_THREAT_MODEL.md
Dashboard counts	FR-DASH-01, FR-DASH-02, FR-ADM-02

12. Future enhancements

Potential later additions include image/media library and featured-image uploads, revision history, scheduling, tags, public author profiles, remember-me sessions, password recovery, trash/restore, view tracking and analytics, comments with moderation, and a more advanced editorial approval workflow. These are not Version 1.0 requirements and must be approved, specified, and tested before implementation.